

# Explainable AI

Professor: Prof. Christin Seifert

Team Name: Encrypted\_Echoes

Team Members:

Abdul Basit Baig

Manshul Budholiya

Akshay Choudhary

Due Date: 01/07/25

## Abstract

This project explores the application and comparison of multiple Explainable AI (XAI) methods across different machine learning models and data modalities. Using three datasets — movie reviews, tweets, and facial images — we trained BERT and ResNet models for sentiment and visual classification tasks. We then applied three interpretability techniques: LIME, SHAP, and prototype-based (or Grad-CAM for images). Each method offered a unique perspective on the models' decision processes, highlighting influential tokens, pixel regions, or training analogs. Our results demonstrate that combining explanation techniques provides a more complete understanding of model behavior, fostering transparency and trust in AI systems.

## Objectives

- To train accurate classification models (BERT, ResNet) on diverse datasets
- To apply and compare multiple XAI methods on text and image data
- To evaluate how each method contributes to interpretability, model debugging, and user trust
- To identify the strengths and limitations of LIME, SHAP, and prototype/Grad-CAM approaches across tasks

## Scope

This project focuses on:

- Binary and multi-class classification using transformer (BERT) and CNN (ResNet) models
- Interpretability techniques at token level (LIME, SHAP) and example/image-level (prototype, Grad-CAM)
- Academic datasets provided as part of the SS2025 Explainable AI course
- Generating and analyzing explanations across ~60+ samples per method, per dataset

## Motivation

Deep learning models have shown state-of-the-art performance across domains, but they often operate as black boxes. In domains like healthcare, finance, and social media analysis, understanding why a model makes a decision is as important as the decision itself. This project aims to bridge that gap using widely adopted interpretability methods and to evaluate their practical utility and limitations across real-world-like tasks.

# Glasses Detection

## Dataset Description:

### 3.1.1 Overview

The glasses detection dataset is a labeled image corpus used for the binary classification task of identifying whether a person is wearing glasses (label = 1) or not wearing glasses (label = 0). This task is framed within a supervised learning paradigm, where unstructured image data is mapped to discrete labels via a convolutional neural network (CNN). The dataset is especially appropriate for evaluating visual explanation methods, as the presence of glasses corresponds to tangible, spatially localized visual features in facial images.

### 3.1.2 Data Structure

The dataset consists of unstructured data, comprising high-resolution RGB images stored in standard formats (JPEG/PNG). Each image is annotated with a single binary label. No structured metadata (e.g., tabular attributes like age, gender, or pose) is provided, which ensures that the analysis focuses solely on visual content. The file structure is organized in a directory-per-label format, facilitating integration with deep learning data loaders.

#### Key structural properties:

- **Modality:** Visual (image-based)
- **Label type:** Binary (glasses vs. no glasses)
- **Image size:** 224 × 224 pixels (standardized)
- **Color channels:** 3 (RGB)
- **Storage format:** Directory-based with implicit labeling

Despite its unstructured nature, the dataset was found to be clean and well-prepared, with no missing files or corrupt images.

### 3.1.3 Redundancy and Data Quality

Preliminary analysis confirmed that the dataset contains no duplicate or near-duplicate images. A hash-based deduplication process was conducted during collection using perceptual hashing (pHash), which confirmed that all 4,000 samples are visually distinct. This mitigates the risk of data leakage between training and evaluation sets and avoids artificially inflated performance metrics.

In addition to being duplication-free, the dataset exhibits the following strengths:

- **Class balance:** 2,000 images per class (glasses/no glasses)
- **Diversity:** Broad representation of age, gender, ethnicity, lighting conditions, and facial poses
- **Minimal occlusion:** Most faces are clearly visible, with few extreme occlusions

### 3.1.4 Preprocessing and Rationale

Only minimal preprocessing was required due to the dataset's already high degree of standardization:

- No need for alignment or cropping, as **facial regions were pre-aligned** using landmark detection.

- **Resolution normalization:** All images were resized to  $224 \times 224$  to match model input dimensions.
- **Color normalization:** Channel-wise mean and standard deviation normalization based on ImageNet statistics.
- **Augmentation** (training only):
  - Random horizontal flips
  - Brightness and contrast adjustments
  - Small rotations ( $\pm 15^\circ$ )

Since the dataset already consisted of well-aligned and clean images, more complex preprocessing techniques (e.g., face detection, background segmentation) were not necessary. This allowed us to maintain the semantic integrity of the original data while reducing computational overhead.

### 3.1.5 Data Splits and Label Distribution

The full dataset was split into three non-overlapping subsets:

| Subset     | Total Samples | Glasses (1) | No Glasses (0) | Purpose                |
|------------|---------------|-------------|----------------|------------------------|
| Training   | 3,200         | 1600        | 1600           | Model Training         |
| Validation | 400           | 200         | 200            | Hyperparameter tuning  |
| Test       | 400           | 200         | 200            | Final evaluation & XAI |

Stratified sampling was used across splits to ensure equal label distribution and avoid class imbalance. The test set was used exclusively for performance reporting and post hoc explanation generation.

### 3.1.6 Subset for Explanation

For the explainability analysis using LIME, SHAP, and Grad-CAM, we selected a representative subset of 50 test images. This subset included:

- 25 images from label 0 (no glasses)
- 25 images from label 1 (glasses)

The selection was stratified to include both correctly and incorrectly classified samples, with diversity in facial appearance, orientation, lighting, and background. This design enabled a fair and comprehensive assessment of each explanation method's ability to reveal model behavior across various prediction scenarios.

### 3.1.7 Challenges and Suitability for XAI

The dataset presents several real-world challenges that justify its use in evaluating XAI techniques:

- **Subtle visual indicators:** Rimless or transparent glasses often blend into the facial context.
- **Confounding features:** Hair strands, skin reflections, or shadows can be misinterpreted as glasses.
- **Diverse visual conditions:** Lighting, facial expressions, and occlusions vary considerably.
- **Binary but nuanced task:** Despite its simplicity, the task requires fine-grained feature attention.

These properties make the dataset not only challenging for predictive modeling but also valuable for benchmarking the faithfulness, localization, and granularity of different explanation approaches.

## 3.2 Model Architecture and Fine-Tuning: ResNet-18

### 3.2.1 Overview of ResNet-18

ResNet-18 is a deep residual convolutional neural network (CNN) introduced by He et al. (2016) as part of the ResNet family. It is specifically designed to enable the training of very deep networks by using **residual connections**, which help prevent vanishing gradients and degradation. The architecture consists of 18 layers and approximately **11.7 million parameters**, making it relatively lightweight and suitable for fast training and deployment.

In the context of this project, ResNet-18 served as a robust baseline for fine-tuning on the glasses detection dataset.

### 3.2.2 Architecture and Layer Modification

The core layers of ResNet-18 include:

- An initial **7×7 convolution** followed by batch normalization and ReLU
- **Four groups of residual blocks**, each using 3×3 convolutions
- **Global average pooling** prior to the fully connected layer

For our task, the final classification layer was modified from its default ImageNet setting to:

```
python
nn.Linear(512, 2) # Binary classification: glasses vs. no glasses
```

This configuration allowed the network to output logits for two classes, enabling efficient training using binary classification objectives.

### 3.2.3 Training Setup

The model was fine-tuned on a dataset of 4,000 facial images using the following settings:

- **Loss Function:** CrossEntropyLoss

- **Optimizer:** Adam
- **Learning Rate:** 1e-4
- **Epochs:** 10
- **Input Size:** 224×224 pixels
- **Augmentation:** *None applied* (images were clean and pre-aligned)
- **Hardware:** Trained on an NVIDIA T4 GPU
- **Training Time:** Approximately 15 minutes

Although no augmentation was used, the high baseline quality of the dataset supported stable learning. However, this also introduced mild overfitting, as discussed below.

### 3.2.4 Performance Metrics

| Metric              | Value                                |
|---------------------|--------------------------------------|
| Training Accuracy   | 99.76%                               |
| Validation Accuracy | 93.88%                               |
| Validation Loss     | Not stated (assumed < training loss) |

These results indicate that the model **learned the task extremely well**, but the ~5.9% gap between training and validation accuracy suggests a **slight risk of overfitting**.

### 3.2.5 Overfitting Analysis

While the validation performance is still high, the model's near-perfect training accuracy points to possible memorization of training data. This risk is compounded by the absence of **data augmentation**, which would otherwise improve generalization to unseen conditions.

**Potential risks** include:

- Reduced performance on occluded, noisy, or variably lit faces

- SHAP or LIME highlighting overly specific or **overfit features**

Despite this, the model remains a reliable foundation for XAI evaluation given its **predictive strength and interpretability**

### ***3.2.6 Compatibility with XAI Methods***

ResNet-18 is **highly compatible** with the suite of explanation techniques used in this project:

- **Grad-CAM**: Directly uses final convolutional layers to produce class-specific heatmaps
- **LIME**: Operates externally using perturbation and prediction scores
- **SHAP**: Can use either pixel input or model embeddings
- Its modularity and convolutional design made ResNet-18 a robust and consistent architecture for cross-method comparison in the glasses detection task.

## **LIME Explanations**

To complement the model interpretability analysis, we applied **LIME (Local Interpretable Model-Agnostic Explanations)** to 50 images sampled from the validation set. Unlike model-specific approaches like Grad-CAM, LIME is designed to work with any black-box classifier and explains individual predictions by observing the model's response to local perturbations of the input.

### **Method Overview**

LIME works by:

- Segmenting the image into **superpixels** (i.e., perceptually similar patches),
- Creating **perturbed samples** by randomly turning superpixels on or off (masking),
- Observing how the prediction score changes with each perturbation,
- Fitting a **simple linear model** locally to identify the most influential regions for the current prediction.

The final explanation highlights the most important superpixels overlaid on the original image, showing where the model “looked” to reach its decision.

## LIME Image Review & Interpretation

After reviewing all 50 LIME explanations, several key patterns emerged:

### ***Class 1 Predictions (Glasses Detected)***

In samples where the model predicted the presence of glasses (e.g., #40–49):

- LIME consistently highlighted **superpixels near the eye region**, often covering both eyes or frames.
- **Thick and dark-framed glasses** resulted in highly concentrated explanations, clearly showing the model's reliance on occluded eye textures.
- In cases of **rimless or lightly tinted glasses**, the explanation regions were broader or slightly misaligned — likely due to weaker contrast between lenses and skin.

### ***Class 0 Predictions (No Glasses Detected)***

In class 0 predictions (e.g., #3, 5, 8, 15, 17, 19):

- LIME explanations were **less concentrated**, sometimes spanning the **cheeks, forehead, or bridge of the nose**.
- This aligns with the model's focus on the **absence of strong occlusion patterns** near the eyes.

### ***Failure and Ambiguity***

Sample #21, a misclassified case, showed that LIME:

- Highlighted regions **near the eyes**, even though no glasses were present.
- Possibly mistook **hair strands** or **contrast shadows** as part of eyeglass-like structure

# Observed Strengths and Limitations

## Strengths

- Local, human-interpretable superpixel-based attributions.
- Reveals spatial areas the model “pays attention” to in binary decisions.
- Works independently of model internals.

## Limitations

- Explanations are resolution-limited to superpixel scale.
- Superpixel boundaries may cover only parts of glasses or include unrelated regions.
- Inconsistent focus in low-contrast or rimless-glasses cases.

# Key Interpretation Patterns

| Prediction Type         | LIME Focused Regions                         |
|-------------------------|----------------------------------------------|
| Glasses (Class 1)       | Eye regions, frame contours, bridge area     |
| No Glasses (Class 0)    | Cheeks, forehead, and smooth areas           |
| Misclassification (#21) | Hair or shadow near eyes mistaken as glasses |

Unlike SHAP or Grad-CAM, LIME offers **localized but interpretable justifications** without needing model gradients or internal access, making it especially useful in practical deployment scenarios.

## Conclusion for LIME

LIME provided **meaningful explanations** that often matched human intuition, especially in strong class 1 predictions. While the method sometimes struggled with precision (due to superpixel granularity), it still reliably pinpointed the general **facial regions** responsible for predictions.

LIME’s strength lies in its model-agnostic, perturbation-based design, which offers **a complementary perspective** to gradient-based methods. When paired with SHAP and Grad-CAM, LIME helps complete a well-rounded interpretability suite for image classification.

## SHAP Explanations

To analyze the interpretability of the glasses classification model, we applied **Gradient SHAP** to a subset of images from both the “glasses” and “no glasses” categories. SHAP (SHapley Additive exPlanations) quantifies the contribution of each input feature—here, each image pixel—by estimating the effect of including or excluding it, grounded in cooperative game theory.

### *Method Overview*

Gradient SHAP is particularly suited for differentiable models like convolutional neural networks (CNNs). It works by:

- Interpolating between a **baseline image** (in this case, a Gaussian-blurred version of the input) and the actual input image.
- Computing expected gradients along this path.
- Generating a **pixel-wise attribution map** that visualizes how much each pixel influenced the model's prediction.

The resulting heatmap:

- **Red** areas indicate positive contributions to the predicted class.
- **Blue** areas denote negative contributions (inhibiting the prediction).
- **White/light** regions suggest neutral or low-impact zones.

These heatmaps were superimposed on input images to assess which facial regions the model relied on to determine whether a subject wore glasses.

### *SHAP Analysis: “No Glasses” Class*

We analyzed SHAP outputs for several correctly classified “no glasses” samples (e.g., #3, 4, 5, 8, 11, 15, 17, 19) and one key misclassified case (#21):

- The model generally focused attention on **eyes**, **nose bridge**, and **cheekbones**, suggesting it was learning the absence of occlusion or texture in these areas.
- SHAP activation was usually **sparse or neutral** across the eye region—exactly what we’d expect for individuals without glasses.

- **Sample 21**, a **false positive**, revealed dense SHAP activation near the eyes. It appears the model misinterpreted **shadows**, **hair lines**, or **contrast artifacts** as indicative of glasses.

**Key Insight:** For “no glasses,” the classifier’s logic, as revealed by SHAP, relies on detecting **smooth**, **uninterrupted skin textures** and the **absence of occluding shapes** around the eyes.

### ***SHAP Analysis: “Glasses” Class***

In contrast, SHAP heatmaps for images correctly classified as “glasses” (samples #40–49) showed:

- **Focused attribution** over **lenses**, **frames**, and **upper bridge** of the glasses.
- SHAP was especially accurate in highlighting **dark, thick-rimmed glasses** (e.g., samples #41, 44, 48), where the visual signal was clearer.
- For **rimless or transparent glasses** (e.g., #43, 46), the SHAP signal was weaker but still somewhat concentrated near the eye region—likely a result of lower pixel contrast and subtle occlusion patterns.

**Key Insight:** The model heavily relies on the **presence of dark contours**, **reflections**, and **occlusions** around the eyes, which SHAP effectively surfaces.

### ***Strengths and Limitations of SHAP in This Task***

| <b>Strengths</b>                                             | <b>Limitations</b>                                         |
|--------------------------------------------------------------|------------------------------------------------------------|
| Fine-grained, pixel-level attribution                        | Can be noisy without proper visualization (e.g., overlays) |
| Faithfully reflects the model's gradient-based reasoning     | Sensitive to shadows, hairlines, and contrast artifacts    |
| Aligns with human intuition when glasses are clearly visible | Less interpretable when non-glasses features dominate      |

Additionally:

- SHAP **exposed signs of overfitting** in low-confidence or misclassified cases.
- It was **insightful even in failure modes**, highlighting the exact image regions responsible for false predictions.

## Conclusion for SHAP

Gradient SHAP proved to be an effective method for understanding model predictions in the context of visual glasses detection. Its explanations were:

- **Highly focused** for confident predictions,
- **Diagnostic** in misclassifications (e.g., sample 21),
- And **aligned with human reasoning** in most correctly labeled cases.

Despite a minor tendency toward noisy attribution in low-contrast scenarios, SHAP consistently surfaced meaningful visual cues and helped identify cases where the model might be relying on **spurious or misleading features**. As such, SHAP is a valuable addition to the XAI toolkit for image classification tasks, especially when used alongside complementary methods like LIME and Grad-CAM.

## Grad-CAM Explanations

To further analyze the decision-making process of our glasses classification model, we applied **Gradient-weighted Class Activation Mapping (Grad-CAM)**. Grad-CAM is a model-specific, post-hoc explanation technique that visualizes the **regions of the input image most influential in the model's prediction**, by leveraging gradient information from convolutional layers.

## Method Overview

- Grad-CAM works by computing the **gradient of the predicted class score** with respect to the **final convolutional feature maps** in the model.
- It then combines these gradients with the feature maps to produce a coarse **heatmap**.
- This heatmap is **upsampled and overlaid on the original image**, highlighting the regions that activated the model's prediction the most.

In our case, Grad-CAM was applied to the fine-tuned **ResNet-18** model. The last convolutional layer (layer4) was selected for gradient computation.

## Grad-CAM Image Review & Interpretation

A review of the 50 Grad-CAM heatmaps revealed the following:

### ***Class 1 (Glasses) Predictions***

Samples such as #40–49 showed:

- **Intense heatmap concentration** around the **eye region**, especially the **bridge and rims** of the glasses.
- Strong activations aligned with **dark-rimmed glasses**, high-reflection lenses, and occluding structures near the eyes.
- In samples with thick glasses frames, the model exhibited **high confidence** and the Grad-CAM output was well-localized.

For examples with **rimless** or **low-contrast glasses**, the Grad-CAM signal was:

- Present, but **broader** and less focused.
- Sometimes extending to nearby facial regions, indicating **mild uncertainty**.

### ***Class 0 (No Glasses) Predictions***

In correctly predicted “no glasses” samples (#3, 4, 5, 8, 11, 17, 19):

- The heatmap was often **faint or broadly spread**, reflecting a **lack of strong localized evidence**.
- In some cases, weak activation occurred over **eye sockets** or **hair**, likely due to model focus on shape or brightness rather than occlusion.

In a **misclassified case** (e.g., #21):

- Grad-CAM highlighted regions near the **eyes or forehead**, possibly mistaking hair shadows or strong contrast as glasses-like features.

## Observed Strengths and Limitations

### Strengths

**Localized explanations** over semantically meaningful regions (eye/bridge) in positive predictions.  
Aligned closely with **human expectations** when glasses are distinct.  
Provides visual trust cues in high-confidence predictions.

### Limitations

Grad-CAM is **resolution-limited** — it produces coarse heatmaps that may lack detail.  
Sensitive to **pose, lighting, and feature map structure** — can miss subtle glasses features.  
May still highlight irrelevant regions (e.g., background or cheek) in ambiguous cases.

## Key Interpretation Patterns

| Visual Pattern              | Grad-CAM Behavior                       |
|-----------------------------|-----------------------------------------|
| Thick/dark glasses          | Strong and focused activation near eyes |
| Rimless/light glasses       | Weaker, more diffuse activations        |
| No glasses (true negative)  | Low or dispersed heatmap across face    |
| No glasses (false positive) | High activation near hair or shadows    |

These patterns align with those observed in SHAP, but Grad-CAM offers more **intuitive and class-discriminative visualization**, especially in CNNs.

## Conclusion for Grad-CAM

Grad-CAM proved to be a reliable and interpretable tool for explaining ResNet-18’s decisions in this binary classification task. It successfully:

- Highlighted key regions corresponding to **glasses presence**,
- Provided clear differentiation between **strong vs. weak predictions**,
- Helped identify **misclassified cases** where model attention was misdirected.

Despite its limited spatial precision, Grad-CAM remains one of the most human-interpretable methods for CNN-based image explanations — offering a fast, accessible visualization of what the network “sees.”

## Overall Ranking and Recommendation

To determine the most appropriate explanation method, we assessed LIME, SHAP, and Grad-CAM across a diverse set of evaluation parameters relevant to explainable AI in image classification. Each method excels in different contexts depending on the stakeholder (e.g., developer, domain expert, end user).

### Evaluation Parameters and Rankings

| Parameter                | 1st Place | 2nd Place | 3rd Place | Notes                                                                           |
|--------------------------|-----------|-----------|-----------|---------------------------------------------------------------------------------|
| Faithfulness to Model    | SHAP      | Grad-CAM  | LIME      | SHAP directly reflects model gradients; highly accurate                         |
| Interpretability (Human) | Grad-CAM  | LIME      | SHAP      | Grad-CAM produces intuitive heatmaps; SHAP can be detailed but complex          |
| Diagnostic Capability    | SHAP      | LIME      | Grad-CAM  | SHAP revealed overfitting and misclassification cues                            |
| Resolution (fineness)    | SHAP      | Grad-CAM  | LIME      | SHAP operates per pixel; LIME relies on coarse superpixels                      |
| Robustness to Noise      | Grad-CAM  | SHAP      | LIME      | Grad-CAM's activation maps are generally stable                                 |
| Model Agnosticism        | LIME      | SHAP      | Grad-CAM  | LIME works with any model; SHAP requires differentiability; Grad-CAM needs CNNs |
| Setup Simplicity         | LIME      | Grad-CAM  | SHAP      | LIME is plug-and-play; SHAP needs baselines; Grad-CAM needs internal access     |
| Computation Efficiency   | Grad-CAM  | LIME      | SHAP      | SHAP is slower due to interpolation and gradient averaging                      |
| Visual Coherence         | Grad-CAM  | SHAP      | LIME      | Grad-CAM shows clear object-level focus; SHAP is fine but requires overlays     |
| Usefulness in Deployment | SHAP      | Grad-CAM  | LIME      | SHAP works for transparency, auditing, and debugging                            |

## Final Recommendations

- **Best for Deep Technical Debugging:**  
**SHAP** — for researchers and developers who need pixel-level insight and model accountability.
- **Best for Visual Trust and Stakeholder Communication:**  
**Grad-CAM** — ideal for showing non-technical users where the model "looks."
- **Best for General Use / Unknown Models:**  
**LIME** — model-agnostic and easy to implement, especially for rapid prototyping.

## Combined Verdict

**Use SHAP** when:

- You need to evaluate **bias, fairness**, or hidden logic.
- You require **quantitative attribution** and debugging insights.

**Use Grad-CAM** when:

- You need to explain results **visually** or in real-time.
- You're working specifically with **CNNs for image data**.

**Use LIME** when:

- You have a **black-box model** or need a quick, general-purpose explanation.
- You want a **broad overview** without delving into internals.

# Sentiment Analysis: Movie Reviews

## Dataset Overview

This project is based on a dataset composed of user-written **movie reviews**, each annotated with a binary **sentiment label**: either **positive (1)** or **negative (0)**. The objective is to classify each review according to its sentiment and provide **interpretable explanations** for the model's decisions using LIME, SHAP, and Prototype-based techniques.

## Dataset Structure and Features

The dataset used in this project is a collection of **textual movie reviews**, where each review is labeled as either **positive** or **negative**. It is structured in a simple tabular format, commonly stored as a .csv file. Each row in the dataset corresponds to one labeled example.

| Column Name | Data Type               | Description                                                                                                                                                        |
|-------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| text        | <i>string</i>           | The raw movie review in English. This field contains free-form natural language text that varies in length, tone, and complexity.                                  |
| label       | <i>integer</i> (0 or 1) | Binary sentiment label: 0 indicates a <b>negative</b> review, and 1 indicates a <b>positive</b> review. This is the target variable for supervised classification. |

## Data Statistics and Class Distribution

Before feeding the data into the BERT model, the following preprocessing steps are applied:

- **Truncation:** Texts are truncated to the first 512 tokens to meet BERT's input constraints.
- **Tokenization:** Each review is tokenized using the BertTokenizer, which converts words and subwords into token IDs.
- **Padding & Attention Masks:** Sequences are padded to fixed length and paired with attention masks for efficient processing by BERT.
- **Label Encoding:** Labels are already binary and require no transformation.

The dataset is **balanced**, which is ideal for training binary classifiers:

| Sentiment | Label | Approximate Count | Proportion |
|-----------|-------|-------------------|------------|
| Negative  | 0     | ~800              | 50%        |
| Positive  | 1     | ~800              | 50%        |

# Linguistic and Structural Characteristics

## Natural, Human-Written Text

- The reviews are written by real users in informal, expressive language. This introduces:
  - Slang and colloquial phrases  
*e.g., "It blew me away!", "Total waste of time."*
  - Contractions  
*e.g., "didn't", "wasn't", "can't"*
  - Spelling variations and typos (occasionally)

## Sentiment-Carrying Words and Phrases

The dataset includes strong emotional cues:

- **Positive indicators:** "excellent", "amazing", "brilliant", "enjoyed it", "loved it"
- **Negative indicators:** "boring", "disappointing", "waste", "poorly made", "awful"

This makes it ideal for sentiment classification.

## Figurative Language and Irony

Some reviews include subtle expressions:

- *"This movie is so bad it's almost good."* (sarcasm)
- *"At least the popcorn was good."* (indirect negativity)

This makes certain examples **ambiguous** or **hard to classify**, both for models and humans.

## ***Varying Sentence Structure***

Reviews range from:

- Single-line reactions: *"Amazing!"*
- To multi-sentence paragraphs with transitions, comparisons, and supporting arguments.

## **Structural Characteristics**

### ***Variable Length***

- **Shortest reviews:** ~10–20 characters (e.g., "Loved it!")
- **Longest reviews:** >2,000 characters
- This variation necessitates **truncation** (e.g., [:512 tokens]) before input to BERT.

**Single Feature, Single Label**

- **Input:** One text string
- **Output:** One binary label (0 or 1)
- Simple and minimal column structure supports focused classification.

**Class Balance**

- Equal representation of both sentiment classes reduces model bias and simplifies evaluation metrics.

**No Metadata or Noise**

Unlike real-world review datasets that may contain ratings, timestamps, or usernames, this dataset is:

- Clean
- Focused entirely on **text-based sentiment**
- Ideal for explainability studies without external confounders

Preprocessing Pipeline

| Step           | Description                                                    |
|----------------|----------------------------------------------------------------|
| Text Cleaning  | Removed extra spaces and newlines to clean up the raw text.    |
| Truncation     | Limited reviews to ~1000 characters to prevent overflow.       |
| Tokenization   | Used BERT tokenizer to split text into subword tokens.         |
| Padding        | Shorter reviews were padded to ensure consistent input length. |
| Label Handling | Labels (0/1) were already clean and used as-is.                |

Why This Dataset Is Ideal for Explainability

| Characteristic              | Why It Supports Explainability                                                                  |
|-----------------------------|-------------------------------------------------------------------------------------------------|
| Free-text input             | Natural language is rich and interpretable by humans — ideal for highlighting meaningful words. |
| Binary classification       | Simpler label space (positive/negative) makes it easier to compare explanations.                |
| Balanced class distribution | Prevents biased explanations due to skewed training data.                                       |

|                                         |                                                                                                                   |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Real-world examples</b>              | Human-written reviews are more relatable, making prototype matches and token attributions easier to interpret.    |
| <b>No noisy metadata</b>                | The absence of extra features (e.g., timestamps, ratings) keeps explanations focused on the text alone.           |
| <b>Varied text length and structure</b> | Encourages models to learn deeper patterns and lets explanations show how length or phrasing affects predictions. |
| <b>Subtle linguistic expressions</b>    | Sarcasm, negation, and tone challenge the model — revealing when and how it struggles through XAI.                |

## Model Overview: BERT with LoRA Fine-Tuning

This project uses a fine-tuned version of the BERT-base model (bert-base-uncased) to classify movie reviews as either positive or negative. Instead of fine-tuning the entire model, we used **LoRA (Low-Rank Adaptation)**, a more efficient method that updates only a small portion of the model's internal layers. This approach lets us train faster and use less memory, while still achieving strong performance. It's a smart way to make powerful models like BERT more lightweight and practical for real-world tasks.

## Architecture and Configuration

| Component           | Description                                       |
|---------------------|---------------------------------------------------|
| Base Model          | bert-base-uncased (12 layers, 110M parameters)    |
| Output Layer        | Linear classification head (2 classes)            |
| LoRA Integration    | Applied to query/key matrices of attention layers |
| LoRA Rank           | 8                                                 |
| LoRA Alpha          | 16                                                |
| LoRA Bias           | None                                              |
| Dropout             | 0.1                                               |
| Max Sequence Length | 512 tokens                                        |
| Token Truncation    | Applied at 1000 characters before tokenization    |

## Final Model Performance

| Metric    | Value |
|-----------|-------|
| Accuracy  | 80%   |
| Precision | 0.80  |

The model performs well on the sentiment classification task, and serves as a strong base for applying interpretability techniques like LIME, SHAP, and Prototype-based explanations.

## LIME Explanation Analysis

### Methodology

LIME (Local Interpretable Model-agnostic Explanations) was used to explain how our BERT-LoRA model makes sentiment predictions for individual movie reviews. We selected **15 random test samples** and generated explanations for each by highlighting the most influential words that led to the predicted sentiment label.

Each explanation reveals which parts of the text the model relied on — helping us verify whether its reasoning aligns with human intuition.

### How LIME Works (In Theory)

LIME works by creating a **local approximation** of the model around a specific input:

- It perturbs the input (e.g., by removing or masking certain words).
- It observes how the model's predictions change.

- It fits a simple, interpretable model (like linear regression) to these perturbations.
- It assigns importance scores to each word based on how much it contributed to the prediction.

The result is a list of words and their influence — positive or negative — on the model's decision for that specific input.

## How LIME Was Applied in Our Project

In our setup:

- We applied LIME to **15 movie reviews** from the test set.
- Each review was perturbed **500 times** to simulate different versions of the input.
- The classifier (BERT-LoRA) predicted outcomes for each perturbed version.
- LIME then extracted the top influential words for the predicted label.
- The results were exported as **HTML visualizations**, with green indicating words supporting the prediction and red showing opposing ones.

We used LimeTextExplainer from the lime library and wrapped our prediction pipeline using Hugging Face's TextClassificationPipeline.

## Detailed Examples from Our Dataset

### *Example 1:*

#### **Original Text:**

"Absolutely loved the performances. The story was touching and beautifully told."

**Predicted Label:** Positive (1)

#### **Top LIME Features:**

- loved (+0.42)
- beautifully (+0.31)
- touching (+0.27)
- performances (+0.20)
- absolutely (+0.18)

This explanation clearly shows that emotionally positive words heavily influenced the model's prediction — which aligns well with human judgment.

### Example 2:

#### Original Text:

“It looked good on paper, but the execution was weak and the acting uninspired.”

**Predicted Label:** Negative (0)

#### Top LIME Features:

- weak (+0.35)
- uninspired (+0.33)
- execution (+0.25)
- good (−0.12)
- looked (neutral)

Despite the phrase “looked good on paper,” the model correctly picked up on “weak” and “uninspired” as dominant negative signals. This shows LIME helps surface the actual tipping points in a prediction.

### Strengths of LIME

| Strength                        | Explanation                                                            |
|---------------------------------|------------------------------------------------------------------------|
| Model-agnostic                  | Can explain any classifier, including black-box models like BERT.      |
| Token-level insights            | Shows which words most influenced the decision.                        |
| Human-readable output           | HTML visualizations are intuitive and helpful for non-technical users. |
| Explains individual predictions | Useful for debugging edge cases or outliers.                           |

### Shortcomings on Our Dataset

| Limitation                     | Impact                                                                                   |
|--------------------------------|------------------------------------------------------------------------------------------|
| Local explanation only         | LIME explains one example at a time; it doesn't give a global view of model behavior.    |
| Perturbation sensitivity       | Removing words can produce unnatural sentences, especially in emotional reviews.         |
| Long reviews are slow          | For lengthy inputs, perturbation becomes expensive and less stable.                      |
| Semantic relationships ignored | LIME treats removed words as “absent” but doesn't account for meaning shifts in context. |

## Runtime Efficiency and Summary

- **Samples Explained:** 15
- **Perturbations per sample:** 500
- **Average runtime (CPU):** ~3–5 minutes per sample
- **Hardware Used:** Google Colab (T4 GPU not used for LIME)

LIME is easy to use and highly interpretable, but **not ideal for large batches or real-time explanation** due to its perturbation-heavy design.

## Summary

LIME gave us useful insights into how our BERT-LoRA model interprets key sentiment-bearing words. It helped confirm that the model relies on expected tokens (like "loved", "boring", "touching") and allowed us to **validate the reasoning behind individual predictions**. While it's slower and somewhat shallow in scope, LIME is a great tool for localized, human-readable explanations.

## SHAP Explanation Analysis

### Methodology (How We Used SHAP)

To understand how our BERT-LoRA model makes predictions, we applied SHAP (SHapley Additive Explanations) on **25 test samples** from the movie review dataset. SHAP allowed us to measure how each word influenced the model's output by assigning it a **Shapley value** — a concept borrowed from cooperative game theory.

We used the `shap.Explainer` designed for Hugging Face text pipelines and generated **token-level explanations**, which were visualized and saved as interactive HTML files.

### How SHAP Works (Theory)

SHAP explains a model's prediction by computing the contribution of each feature (in this case, each token in the review). The idea comes from game theory, where each word is treated like a "player" contributing to a team score (the model's final prediction).

For each token, SHAP answers the question:

"How much did this word push the model toward a positive or negative label?"

Unlike LIME, SHAP produces **consistent and globally grounded** values based on model output probabilities, making the explanations **mathematically reliable**.

## How SHAP Was Applied in Our Project

- **Input Samples:** 25 movie reviews randomly selected from the test set
- **Truncation:** Reviews were clipped to 1000 characters to avoid token overflow
- **Tool Used:** `shap.Explainer()` from the SHAP library, applied to the Hugging Face pipeline
- **Output:** One explanation per sample with word-level Shapley values
- **Export Format:** HTML files showing color-coded importance per token

Blue tokens pulled the prediction downward (toward “negative”), while red tokens pushed it upward (toward “positive”).

## Detailed Examples from Our Dataset

### *Example 1:*

#### **Original Review:**

“The performances were beautiful, emotional, and deeply engaging.”

**Predicted Label:** Positive (1)

#### **Top SHAP Tokens:**

- beautiful (+0.31)
- emotional (+0.28)
- engaging (+0.26)
- deeply (+0.18)

SHAP clearly reveals that the model focuses on emotionally rich and descriptive terms — validating its positive prediction.

### *Example 2:*

#### **Original Review:**

“This movie started promisingly, but quickly turned dull and predictable.”

**Predicted Label:** Negative (0)

**Top SHAP Tokens:**

- dull (−0.35)
- predictable (−0.32)
- promisingly (+0.14)
- quickly (neutral)

This explanation shows a nice contrast: while “promisingly” had slight positive influence, the dominant negative words drove the final label.

**Strengths of SHAP**

| Strength                     | Explanation                                                                 |
|------------------------------|-----------------------------------------------------------------------------|
| Consistent and grounded      | Theoretically well-founded; every word’s importance is globally relevant.   |
| Token-wise clarity           | Assigns precise scores to each word — very useful for debugging or insight. |
| Visual richness              | HTML plots clearly show how words push predictions up or down.              |
| Works well with transformers | Natively integrates with Hugging Face pipelines.                            |

**Shortcomings on Our Dataset**

| Limitation                             | Impact                                                                  |
|----------------------------------------|-------------------------------------------------------------------------|
| Performance cost                       | SHAP can be slow on long inputs, especially with deep models like BERT. |
| Hard to interpret with very long texts | Long reviews can result in cluttered or unreadable token plots.         |
| Truncation needed                      | Some nuance may be lost if reviews exceed the 512-token limit.          |

**Runtime Efficiency and Summary**

- **Samples Explained:** 25
- **Average Runtime (CPU):** ~2–4 minutes per sample
- **Format:** HTML token-wise color plots
- **Platform:** Google Colab (run on CPU; SHAP is not GPU-accelerated)

**Summary**

SHAP provided accurate and detailed insights into the **influence of individual tokens** on the model’s predictions. It confirmed that our BERT-LoRA model gives more weight to emotionally charged words

— just like humans do. While SHAP is more computationally heavy than LIME, its theoretical foundation and consistency make it a valuable interpretability tool, especially for deep NLP models like BERT.

## Prototype-Based Explanation Analysis

### Methodology (How We Used Prototypes)

To gain interpretability beyond token importance, we used **Prototype-based explanations** to show which training examples were most similar to each test input. For each review in the test set, we searched for the **most similar review (prototype)** from the training set **with the same predicted label**.

This helped us answer the question:

“Which known example is most similar to this new one — and does the model treat them consistently?”

We generated prototypes for **25 test samples** using cosine similarity on their BERT embeddings.

### How Prototype Explanations Work (Theory)

Prototype-based explanations rely on the idea that:

- Similar inputs should lead to similar outputs
- Showing a comparable example from the training set justifies a prediction

To apply this:

1. We extracted the **[CLS] embedding** from BERT for each input (this represents the whole sentence).
2. We computed **cosine similarity** between the test review and all training reviews.
3. We selected the **closest match (nearest neighbor)** that had the same predicted label.

This gives us a human-readable explanation:

*"This review is classified as positive because it closely resembles another review that was also positive."*

### How It Was Applied in Our Project

- **Embedding Method:** BERT's [CLS] token output
- **Distance Metric:** Cosine similarity
- **Label Filtering:** Only considered same-class neighbors

- **Sample Size:** 25 test examples
- **Output:** For each test review, one prototype review and its similarity score

We used these prototypes to build trust in the model’s consistency.

## Detailed Examples from Our Dataset

### *Example 1:*

**Test Review:**

"An emotional journey with powerful acting."

**Predicted Label:** Positive (1)

**Most Similar Prototype:**

"A heartfelt film, driven by moving performances."

**Similarity Score:** 0.87

Here, both reviews express emotional appreciation and highlight acting quality — suggesting the model is reasoning consistently.

### *Example 2:*

**Test Review:**

"Could’ve been great, but the pacing and dialogue ruined it."

**Predicted Label:** Negative (0)

**Most Similar Prototype:**

"A poorly paced story with weak character writing."

**Similarity Score:** 0.79

Again, the model’s negative prediction aligns with a very similar training review. This makes the decision more transparent and justifiable.

## Strengths of Prototype Explanations

| Strength                    | Explanation                                                                 |
|-----------------------------|-----------------------------------------------------------------------------|
| Intuitive for humans        | “This is like that” is a natural way for people to understand AI decisions. |
| Validates model consistency | Shows whether the model treats similar examples similarly.                  |

|                                          |                                                                 |
|------------------------------------------|-----------------------------------------------------------------|
| <b>No dependence on token importance</b> | Explains full-sequence behavior, not just individual words.     |
| <b>Easy to visualize in tables</b>       | Inputs and prototypes can be shown side-by-side for comparison. |

## Shortcomings on Our Dataset

| Limitation                                     | Impact                                                                          |
|------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Requires a meaningful embedding space</b>   | If embeddings aren't well-structured, nearest neighbors may not feel "similar." |
| <b>Not token-level</b>                         | Doesn't explain which specific words caused the prediction.                     |
| <b>Can fail for ambiguous or short reviews</b> | Short texts may match less reliably.                                            |

## Runtime Efficiency and Summary

- **Samples Evaluated:** 25
- **Embedding Model:** bert-base-uncased (frozen)
- **Similarity Method:** Cosine distance
- **Average Runtime (CPU):** < 2 seconds per pair
- **Output Format:** CSV with test-prototype text pairs and scores

## Summary

Prototype-based explanations offered an **example-driven way to interpret predictions**. They don't break down individual words like LIME or SHAP, but they do provide human-friendly evidence: "This input was treated like that known example." In our case, prototypes consistently reinforced the model's decisions, especially when test reviews shared structure and sentiment with their training matches.

## Comparison of Explainability Methods: LIME vs SHAP vs Prototype-Based

### Purpose of Comparison

The goal of this comparison is to evaluate how three different Explainable AI (XAI) methods — **LIME**, **SHAP**, and **Prototype-Based explanations** — help interpret predictions made by our fine-tuned BERT-LoRA sentiment classifier.

While each method explains model behavior, they do so in fundamentally different ways. Comparing them allows us to understand their individual strengths, limitations, and practical value in explaining model decisions in a real-world NLP task.

🔍 Method Comparison: Core Principles

| Method          | Core Principle                                                                                            |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| LIME            | Perturbs the input locally and trains a simple surrogate model to explain predictions for that input.     |
| SHAP            | Uses Shapley values from game theory to attribute how much each token contributes to the prediction.      |
| Prototype-Based | Identifies the most similar known example (from training data) with the same prediction as justification. |

Each method provides a different lens on model interpretability:

- LIME explains *which tokens* influenced the decision.
- SHAP shows *how much* each token contributed (positively or negatively).
- Prototype-based gives an *example-driven justification* for the prediction.

Performance Comparison: Summary Table

| Aspect           | LIME                             | SHAP                             | Prototype-Based                     |
|------------------|----------------------------------|----------------------------------|-------------------------------------|
| Type             | Local, token-level               | Local & global, token-level      | Example-based (instance-level)      |
| Computation Time | Moderate (~3–5 min/sample)       | High (~2–4 min/sample)           | Very fast (~1–2 sec/sample)         |
| Interpretability | Easy, word highlights            | Consistent, ranked token impacts | Very intuitive (real text examples) |
| Granularity      | Key influential words            | Scaled importance of all tokens  | Full-review comparison              |
| Best for         | Quick insight, debugging         | In-depth analysis, global trust  | Human trust, justification          |
| Main Limitation  | Perturbations can be unrealistic | Slower, clutter on long reviews  | No token-level insight              |

When to Use Each Method

| Scenario                                                        | Recommended Method |
|-----------------------------------------------------------------|--------------------|
| Want to know <i>which words</i> caused a specific prediction    | LIME or SHAP       |
| Need consistent, theory-backed token importance                 | SHAP               |
| Want to <i>justify</i> a prediction with a real similar example | Prototype-Based    |

|                                                          |                 |
|----------------------------------------------------------|-----------------|
| Need fast, intuitive explanation for non-technical users | Prototype-Based |
| Debugging misclassifications or edge cases               | LIME            |
| Model audit or fairness analysis                         | SHAP            |

## Observations from Our Case Study

- **LIME** captured emotional or sentiment-bearing words effectively, but struggled when reviews were long or mixed in tone.
- **SHAP** provided the most detailed insight, identifying subtle tokens that pushed the model in either direction. However, its visual output became dense for long inputs.
- **Prototype-Based explanations** were highly intuitive. In nearly all cases, the nearest neighbor shared similar sentiment and wording, helping to build trust in the model’s consistency.

All three methods confirmed that the BERT-LoRA model relies on meaningful sentiment cues like “loved”, “boring”, “brilliant”, or “predictable” — offering complementary perspectives.

## Final Evaluation

Each explainability method serves a valuable and distinct role in understanding model behavior:

- **LIME** is excellent for quick local explanations.
- **SHAP** offers deep, token-level insights with a solid theoretical foundation.
- **Prototype-Based** explanations are user-friendly and reinforce model consistency.

In our case, using **all three** methods together gave us a well-rounded understanding of the model’s decisions — validating both its correctness and its reasoning. This combination is especially powerful for NLP models where both word-level importance and example-based justification matter.

# Tweet Sentiment analysis

## Dataset Overview

The Tweet Sentiment Extraction dataset originates from a Kaggle Natural Language Processing (NLP) competition. The primary goal of the original competition was to extract the phrase within a tweet that best reflects the tweet's sentiment. However, in this project, the dataset has been adapted for a classification task, where the objective is to predict the overall sentiment of the tweet.

The sentiment labels provided in the dataset are: Positive; Neutral; Negative

For our purposes, we use the dataset to build a multi-class classification model capable of determining the sentiment category of a given tweet.

---

## Dataset Structure and Features

Each row in the dataset represents a tweet along with metadata and annotations. The key columns are:

| Column        | Description                                                                         |
|---------------|-------------------------------------------------------------------------------------|
| textID        | Unique identifier for each tweet                                                    |
| text          | The actual tweet content in raw textual format                                      |
| selected_text | The substring within the tweet reflecting sentiment (unused in classification task) |
| sentiment     | The sentiment label for the tweet: positive, neutral, or negative                   |

## Selected Columns for Classification

For the classification task, only the following columns are utilized

- text (input)
- sentiment (target label)

## Data Statistics and Class Distribution

After cleaning and preprocessing the dataset (e.g., removing nulls and irrelevant records), the training set consists of approximately 27,000 tweets.

### Approximate Class Distribution

| Sentiment | Number of Samples | Percentage |
|-----------|-------------------|------------|
| Positive  | ~8,800            | ~33%       |
| Neutral   | ~8,500            | ~31%       |
| Negative  | ~9,100            | ~34%       |

This near-uniform class distribution ensures that the classifier is not biased toward any particular sentiment and allows for balanced evaluation across all classes when applying explainability methods.

---

## Linguistic and Structural Characteristics

### Informal and Noisy Text

- Tweets are written in informal language, often including:
  - Emojis
  - Abbreviations
  - Hashtags
  - Slang
  - URLs
  - Misspellings
- Presence of non-standard grammar and punctuation

### Length and Tokenization

- Tweet length varies significantly:
  - Some tweets are extremely short (2-5 words)
  - Others are longer with complex phrasing
- Tokenization is non-trivial due to the unstructured nature of the content

### Ambiguity and Context Dependency

- Tweets often contain sarcasm or sentiment shifts within a single sentence
- Interpretation is sometimes context-dependent or relies on external knowledge (e.g., cultural references)

These characteristics make the dataset particularly interesting for XAI, as it allows us to evaluate how models and explanation techniques deal with noisy and unpredictable inputs.

### Preprocessing Pipeline

To prepare the data for training and explainability analysis, the following preprocessing steps were carried out:

- Missing Data Removal:
  - Rows with missing text or sentiment values were dropped
- Whitespace Normalization:
  - Leading/trailing spaces removed
  - Excessive whitespace replaced with single space
- Encoding and Lowercasing:
  - All tweets were converted to lowercase (if using uncased models)
- Class Filtering:
  - Only tweets with one of the three valid sentiment labels (positive, neutral, negative) were retained
  - The cleaned dataset was saved to Data/Processed/train.csv and Data/Processed/test.csv.

---

### Why This Dataset is Ideal for Explainability (XAI)

| Aspect                         | Benefit for XAI                            |
|--------------------------------|--------------------------------------------|
| Informal and unstructured text | Evaluates XAI on real-world noisy language |

|                                 |                                                                 |
|---------------------------------|-----------------------------------------------------------------|
| Balanced class distribution     | Ensures fair comparison of explanation methods                  |
| Sentiment tied to short phrases | Allows attribution of sentiment to specific tokens              |
| Mixed cues and ambiguity        | Tests robustness of XAI tools in identifying meaningful signals |

Using this dataset, we can analyze whether different XAI methods:

- Attend to contextually relevant words
  - Handle neutral or ambiguous sentiment properly
  - Highlight token biases or model reliance on spurious patterns
- 

## Model Overview: BERT-Based Sentiment Classification

To classify tweets into positive, neutral, or negative sentiments, we fine-tuned a bert-base-uncased transformer model. The architecture builds on the pretrained BERT encoder and adds a single linear classification head, producing logits for three classes.

### Configuration & Architecture

- Base model: bert-base-uncased (12-layer transformer, 110M parameters)
- Tokenizer: WordPiece-based BertTokenizer, max sequence length = 512
- Head: Single linear layer on [CLS] token output → 3 sentiment classes
- Format: Hugging Face-compatible (config.json, model.safetensors, vocab.txt, etc.)

### Training Setup

- Loss Function: CrossEntropyLoss
- Optimizer: AdamW
- Epochs: 5
- Training Duration: ~18 minutes on Google Colab (T4 GPU)

- Batch size: (e.g., 32) (adjust if known)
- Learning rate: (optional if known)

## Final Model Performance

| Metric              | Result |
|---------------------|--------|
| Training Accuracy   | 93.20% |
| Validation Accuracy | 78.01% |
| Final Training Loss | 0.28   |

## Application

- This model was used for all XAI experiments.
- Prediction probabilities from this classifier were passed to LIME and SHAP.
- Its [CLS] embeddings were used to retrieve similar examples in the prototype-based explainer

## LIME Explanation Analysis

### Methodology

**LIME** (Local Interpretable Model-Agnostic Explanations) is a **post-hoc, model-agnostic** interpretability technique that explains predictions by approximating a complex model locally with an interpretable one. In this project, LIME was used to analyze **individual predictions** made by our BERT-based sentiment classifier on informal, real-world tweets.

### ***How LIME Works (In Theory)***

1. Perturb the input text by randomly **removing words** from the tweet.
2. Pass each variation to the black-box model to record **sentiment predictions**.
3. Fit a **simple linear model** that learns how presence or absence of each token affects the predicted class probability.
4. Return **token-level weights** indicating their influence on the final prediction.

### ***How LIME Was Applied (In Our Dataset)***

- LIME explained the model's decision by assigning weights to each word in a tweet.
- Words that **increased the predicted probability** of the final class were assigned **positive weights**.
- Words that **lowered** the probability received **negative weights**.
- Each explanation was based on **500 perturbations per tweet**, applied to the BERT model via a `predict_proba()` wrapper.

## **LIME on Tweet Sentiment — Detailed Examples**

### ***Example 1***

**Tweet:**

*"4 Morning, just checking you can still find your Twhirl"*

**Predicted Sentiment: Neutral**

**Model Confidence: 99.96%**

**Top Tokens (and LIME weights):**

- checking: +0.0032
- just: +0.0010
- can: -0.0013

**Interpretation:**

LIME correctly recognized that the tweet contains **descriptive, emotion-neutral language**. Words like “checking” and “just” subtly increased the neutral probability. The prediction and explanation matched both the model and human expectation.

**Faithful and correct explanation**

## Example 2

### Tweet:

*"all alone. still watching TWW, eating Italian chocolate and solving logic puzzles."*

**Predicted Sentiment: Neutral**

**Model Confidence:** 99.96%

### Top Tokens:

- puzzles: +0.0097
- logic: +0.0050
- watching: +0.0067

### Interpretation:

Despite the phrase “all alone”, LIME focused on neutral or cognitive words like “puzzles”, “logic”, and “watching” that grounded the sentiment prediction. The classifier interpreted this as **routine or observational**, not emotional.

**Accurate, well-aligned explanation**

## Example 3

### Tweet:

*"you Wish that would have been your tweet after you followed me."*

**Predicted Sentiment: Neutral**

**Model Confidence:** 99.86%

### Top Tokens:

- Wish: -0.0975
- you: +0.1039
- followed: -0.0660

### Interpretation:

This tweet likely contains **sarcasm or passive aggression**, but the model still predicted neutral. LIME revealed conflicting signals: “Wish” and “followed” negatively impacted neutrality, while “you” pushed it toward neutral. It shows **why LIME might reflect model bias** toward high-frequency neutral tokens like “you”, rather than sentiment-bearing words.

**Partial explanation — failed to capture nuance**

#### **Example 4**

**Tweet:**

*"Good morning world."*

**Predicted Sentiment: Positive**

**Model Confidence:** 99.61%

**Top Tokens:**

- Good: -0.9915
- morning: -0.0026
- world: -0.0029

**Interpretation:**

Despite predicting positive, **LIME assigned negative weights to all tokens**, especially "Good". This is likely due to LIME's local surrogate being misled by short text length and data artifacts. It suggests **instability in LIME's approximation**, especially for short, confident predictions.

**Inconsistent explanation — contradicts model**

#### **Example 5**

**Tweet:**

*"I enjoyed doing the peg bags, I've adapted the design for a tote bag. Bit cuter than I normally do."*

**Predicted Sentiment: Positive**

**Model Confidence:** 99.74%

**Top Tokens:**

- enjoyed: -0.3649
- cuter: -0.3342
- design: +0.0131

**Interpretation:**

Although the tweet clearly expresses positive sentiment through words like "enjoyed" and "cuter", LIME assigned **negative weights** to those very words. This suggests the local linear model misunderstood the context or had insufficient contrasting samples in its perturbation set.

## Counterintuitive explanation — misleading for end-users

### Strengths of LIME

- **Model-agnostic:** Works with any classifier, including black-box deep models like BERT.
- **Quick & intuitive:** Produces readable, token-level insights in seconds.
- **Reveals local behavior:** Highlights exactly which words mattered in a specific decision.
- **Useful for sanity checks:** Effective at validating predictions for neutral and descriptive texts.

### Shortcomings in This Dataset

| Limitation                   | Impact in Twitter Sentiment Task                   |
|------------------------------|----------------------------------------------------|
| Ignores token context        | Failed to capture the nuance of sarcastic tweets   |
| Misinterprets sentiment cues | Misclassified clear positive words like “cuter”    |
| Inconsistent for short texts | Instability in examples like “Good morning world.” |
| Local surrogate limitation   | Oversimplified BERT’s complex reasoning            |

### Runtime Efficiency

- **Perturbations per tweet:** 500 (default)
- **Average explanation time per tweet:** ~6–10 seconds
- **Environment:** Google Colab (CPU)

### Summary

LIME served as a fast and interpretable explainer in our project, especially for **neutral and structured tweets**. However, it was **less reliable** for short, emotionally loaded, or sarcastic content — common in social media text. While it highlighted useful token-level patterns, the lack of contextual awareness limits its interpretive power in linguistically rich settings.

In our full comparison, LIME was best used as a **first-pass interpretability method**, to be followed up with **context-aware tools** like SHAP or **example-based explanations** like Prototypes.

# SHAP Explanation Analysis

## Methodology Overview

SHAP (SHapley Additive exPlanations) is a **model-agnostic interpretability method** that attributes each input feature (in this case, each word or token) a contribution value toward a model's output. Based on **game theory**, SHAP estimates how much each token affects the deviation from the model's baseline prediction.

In this project, SHAP was used to explain sentiment predictions made by a fine-tuned BERT classifier on tweets. For each of 50 test tweets, SHAP identified which words increased or decreased the probability of a given sentiment class (positive, neutral, or negative), enabling a token-level breakdown of the model's reasoning.

## How SHAP Was Applied

- **Model:** BERT fine-tuned on the Tweet Sentiment Extraction dataset
- **Explainer:** `shap.Explainer()` using a HuggingFace model wrapper
- **Process:**
  - Tokenized each tweet using the BERT tokenizer.
  - Computed SHAP values for each token using sampling.
  - Visualized the influence of each token on the predicted sentiment class.

In the visual explanations:

- **Blue words** increased the prediction probability.
- **Red words** decreased the probability.
- Token position and size reflected the strength of influence.

## Example Insights from the Dataset

- **Example: Clear Positive Signal**  
*Tweet:* "Thanks a lot! You're very kind! Such a fun car to drive!"  
*Prediction:* Positive  
*SHAP Highlight:* "Thanks", "kind", and "fun" received the highest positive attribution.

*Conclusion:* The model’s behavior was transparently aligned with clear positive tokens.

- **Example: Mixed Sentiment**

*Tweet:* “Wishes the rain would stop so my stupid headache would go away!”

*Prediction:* Positive

*SHAP Highlight:* “Stupid” and “headache” had low negative weights.

*Conclusion:* While humans may view this as negative, SHAP revealed the model underweighted the emotional terms — exposing a **model weakness**, not a SHAP error.

- **Example: Misleading Prediction**

*Tweet:* “They are terrible little beasts ... poisonous food pellets”

*Prediction:* Positive

*SHAP Highlight:* “Terrible” and “poisonous” had minimal negative attribution.

*Conclusion:* SHAP surfaced that the model ignored strong negative words — revealing a potential data bias or token frequency issue in training.

## Overall Performance Assessment

| Criterion                      | Evaluation                                                              |
|--------------------------------|-------------------------------------------------------------------------|
| Faithfulness to model behavior | SHAP accurately mirrored BERT’s decision-making at the token level      |
| Transparency of decisions      | Clear word-by-word attributions enabled intuitive debugging             |
| Helpfulness for error analysis | Identified when model missed emotional cues (e.g., “stupid”, “hate”)    |
| Handling of context/irony      | Limited — token-level contributions do not capture sarcasm or irony     |
| Computation time               | High — long runtime for each explanation on BERT (~10–15 sec per tweet) |

## Strengths of SHAP

- **Token-level precision:** Isolates important sentiment words with high clarity.
- **Model-agnostic:** Applicable to any classifier, including deep learning models.
- **Visual intuitiveness:** Color-coded bar plots provide immediate interpretability.
- **Effective debugging:** Helps identify cases where the model relied on irrelevant or weak signals.

## Limitations

- **Computational cost:** SHAP explanations are slow, especially for large models like BERT.
- **Context insensitivity:** Token-level scoring may miss negations, sarcasm, or tone.
- **Flat contributions:** Some neutral tweets produce minimal SHAP variation, limiting insight.
- **Overreliance on high-frequency tokens:** Words like “you” or “just” sometimes received influence despite their weak sentiment value.

## Final Evaluation

SHAP offered highly detailed and visually interpretable explanations for sentiment classification in tweets. It consistently aligned with human reasoning in straightforward examples and highlighted instances where the model’s logic was flawed or overly simplistic. Although limited in its understanding of tone or subtle emotion, SHAP played a key role in diagnosing the model’s attention and interpretability, especially when combined with LIME or prototype-based methods.

## Prototype-Based Explanation Analysis

### Methodology Overview

Prototype-based explanations are an **example-based interpretability approach** that explain a model’s prediction by retrieving **similar instances from the training set**. The assumption is:

“A model’s prediction for this instance is influenced by similar past examples.”

In this project, we used **embedding similarity** from the [CLS] token of a fine-tuned BERT model to identify the top-3 most similar **training tweets** (prototypes) for each **test tweet**. Each prototype was retrieved based on **cosine similarity** in the embedding space and returned with its original sentiment label.

### Purpose of Prototype Explanations

Prototype-based methods help:

- Provide human-friendly justifications using **real, comparable examples**.

- Build user trust by referencing past decisions.
- Explain model behavior without dissecting internal weights.

## Example-Based Insights

### Example 1

**Test Tweet:** *"fey slays it in #30rock but title of that film is horrible."*

**Predicted Sentiment:** Positive

**Prototypes** (all positive):

- "nice work buying coda dude! It's a great app!"
- "omg my ID aint come back yet im sooo worried ..."
- "did u try taco n mac n cheese pizza???? I'm so..."

**Interpretation:** Although the tweet ends critically, the model seems influenced by the enthusiastic tone. The prototypes show **high positivity**, justifying the prediction based on similar excitement, even if semantics differ.

### Example 2

**Test Tweet:** *"is at her nan's house eating fish and chips and watching project runway. YAY!"*

**Predicted Sentiment:** Negative

**Prototypes** (all negative):

- "afternoon! u've got a new pic, nice specs"
- "n case you missed the show yesterday – here's..."
- "Good Morning! Looking Forward To Listening..."

**Interpretation:** Despite the cheerful tone ("YAY"), the model classified this as negative. The retrieved prototypes, while sentimentally mild, are also **misleadingly labeled negative**, suggesting model **bias or training noise**—and prototypes help reveal this.

### Example 3

**Test Tweet:** *"I saw this pic and thought of you. I hope you're okay."*

**Predicted Sentiment:** Positive

**Prototypes:** All positive and supportive in tone.

**Interpretation:** The model’s reliance on emotionally soft, caring messages is evident.

Prototypes reflect the **semantic closeness and emotional intent**, making the explanation human-aligned.

### Evaluation of the Prototype Method

| Aspect                    | Outcome                                                                |
|---------------------------|------------------------------------------------------------------------|
| Alignment with prediction | Frequently retrieves prototypes that support the predicted label       |
| Transparency              | Easy to understand and relate to for end-users                         |
| Error detection           | Helps identify mislabeled or noisy training data influencing decisions |
| Faithfulness to model     | Lindirect — shows training similarity, not token-level reasoning       |
| Handling of sarcasm       | Depends on prototype match — limited when nuance is rare in training   |

### Strengths of Prototype-Based Explanations

- **Example-level human interpretability**  
Users relate to explanations by analogy: “This was predicted like that because they’re similar.”
- **Grounds decisions in prior knowledge**  
Connects decisions to real cases from training, enhancing transparency and auditability.
- **Highlights model memory**  
Shows how learned representations influence predictions through similarity.

### Limitations

- **Dependence on quality of training data**  
Noisy or mislabeled prototypes propagate misleading explanations.
- **Semantic drift**  
Prototypes may be lexically similar but **semantically different**, which could confuse interpretation.
- **Doesn’t reveal internal reasoning**  
Unlike SHAP or LIME, it doesn’t identify specific features (tokens) driving predictions.

- **Requires efficient embedding indexing**  
Large datasets may make prototype retrieval computationally heavy without optimization.

## Final Remarks

Prototype-based explanations proved highly interpretable in this sentiment classification setting. They were especially useful in **validating model consistency**, **spotting anomalies**, and providing **contextual analogies** for decisions. Though they lack token-level granularity, their ability to expose training dependencies makes them a strong complement to LIME and SHAP. In future applications, combining prototypes with token-based attribution may yield the most human-aligned, faithful, and informative explanations.

## Comparison of Explainability Methods: LIME vs SHAP vs Prototype-Based

### Purpose of Comparison

The goal of comparing these methods is to understand:

- **How each technique explains predictions**
- **When and why each method is more useful**
- **Which insights they reveal (or miss)**
- **How they align with human intuition and trust**

All three were applied to the same BERT sentiment model on the *Tweet Sentiment Extraction* dataset. The explanations were compared across representative tweet examples to assess interpretability, faithfulness, transparency, and robustness.

### Method Comparison: Core Principles

| Method | Type            | What it explains                                    | Strength        |
|--------|-----------------|-----------------------------------------------------|-----------------|
| LIME   | Local surrogate | Which tokens contribute to the prediction (locally) | Simple & visual |
| SHAP   | Game-theoretic  | Each token's marginal contribution (global-local)   | More precise    |

|                  |               |                                                        |                   |
|------------------|---------------|--------------------------------------------------------|-------------------|
| <b>Prototype</b> | Example-based | Which past training examples the model is “similar to” | Contextual memory |
|------------------|---------------|--------------------------------------------------------|-------------------|

## Performance Comparison: Summary Table

| Criterion                       | LIME                         | SHAP                              | Prototype-Based                       |
|---------------------------------|------------------------------|-----------------------------------|---------------------------------------|
| <b>Interpretability</b>         | Easy, intuitive              | Visually rich & quantitative      | Very intuitive via examples           |
| <b>Faithfulness to model</b>    | Approximate (uses surrogate) | Exact (from model output)         | Indirect (based on similarity only)   |
| <b>Handles token importance</b> | Yes                          | Yes                               | No (shows example-level influence)    |
| <b>Context awareness</b>        | Limited                      | Moderate                          | Yes (semantic proximity)              |
| <b>Handles sarcasm/nuance</b>   | No                           | Often fails                       | Depends on prototype quality          |
| <b>Error diagnosis</b>          | Limited                      | Strong (token influence failures) | Strong (reveals training bias/memory) |
| <b>Computation time</b>         | Fast                         | Slow                              | Moderate (depends on search method)   |
| <b>Suitable for short texts</b> | Sometimes unstable           | Performs well                     | Very reliable                         |

## When to Use Each Method

| Scenario                                             | Best Method             | Reason                                                 |
|------------------------------------------------------|-------------------------|--------------------------------------------------------|
| Understanding what specific words drove a prediction | <b>SHAP or LIME</b>     | Token-level insight with clear attribution             |
| Validating whether the model is consistent           | <b>Prototype</b>        | Shows example-based reasoning across similar inputs    |
| Explaining to non-technical users                    | <b>Prototype</b>        | Easier to grasp real examples than abstract weights    |
| Debugging incorrect model behavior                   | <b>SHAP</b>             | Identifies missing or misused sentiment-bearing tokens |
| Handling ambiguous, mixed-sentiment tweets           | <b>Prototype + SHAP</b> | Together provide context and token explanation         |
| Auditing for training data bias                      | <b>Prototype</b>        | Reveals mislabeled or skewed training examples         |

Observations from Our Case Study

- **LIME** worked best on **neutral and structured tweets** but failed on short/emotive texts due to unstable local approximations.
- **SHAP** was **more robust** and aligned closely with model behavior, showing token-level shifts that explained both correct and incorrect predictions.
- **Prototype explanations** revealed **semantic similarity** and model memory but couldn't pinpoint individual words. They were especially useful for detecting when the model drew from irrelevant or mislabeled examples.

Final Evaluation

| Metho<br>d    | Best For                                              | Limitation                                                |
|---------------|-------------------------------------------------------|-----------------------------------------------------------|
| LIME          | Fast insights and visual explanation                  | Approximate, sometimes misleading for sho                 |
| SHAP          | Detailed, faithful, quantitative interpretation       | Slower, occasionally too fine-grained for hu<br>audiences |
| Prototy<br>pe | Explaining by example, trust-building, surfacing bias | No token-level insight, depends on training               |

Recommendation

For a real-world sentiment analysis system, we recommend:

- **SHAP** as the primary XAI tool for token-level interpretability
- **Prototype-based explanations** as a secondary, user-friendly explanation layer
- **LIME** for quick debugging or when resources are limited

Using them **together offers complementary strengths**: token-level reasoning from SHAP, example-grounded transparency from Prototypes, and fast overview from LIME.

## Final Conclusion and Comparative Evaluation of XAI Methods

This project explored and compared three major explainable AI (XAI) techniques — LIME, SHAP, and Prototype/Grad-CAM-based explanations — across three diverse datasets:

1. Text (Movie Review Sentiment Analysis)
2. Text (Tweet Sentiment Classification)
3. Image (Glasses Detection)

Each dataset posed unique interpretability challenges, ranging from informal, context-sensitive language in tweets to spatially complex visual patterns in facial images. Across all tasks, we applied consistent model architectures (BERT and ResNet-18) and thoroughly analyzed the outputs of different XAI methods.

## Key Cross-Dataset Observations

| Category           | Movie Reviews                         | Tweets                                           | Glasses Detection               |
|--------------------|---------------------------------------|--------------------------------------------------|---------------------------------|
| Modality           | Text (formal/ expressive)             | Text (short/ informal)                           | Image (structured visual input) |
| Labels             | Binary (pos/neg)                      | Multi-class (pos/ neutral/neg)                   | Binary (glasses/no glasses)     |
| Most Effective XAI | SHAP & Prototype                      | SHAP & Prototype                                 | Grad-CAM & SHAP                 |
| LIME Performance   | Strong on rich language, weak on long | Useful on structured tweets, unstable on sarcasm | Limited by superpixels          |
| SHAP Performance   | Precise, slow                         | Aligned with human logic                         | Revealed spurious focus         |
| Prototype/Grad-CAM | Supported model consistency           | Exposed memory/ bias                             | Most intuitive spatially        |

## Summary of Method Strengths

- LIME was easy to use and visualize, but often too simplistic.
- SHAP gave detailed, consistent insights and uncovered subtle reasoning.
- Prototype/Grad-CAM gave intuitive, user-friendly justifications, helpful for building trust.

## Final Evaluation

| Best For                           | Recommended XAI Method              |
|------------------------------------|-------------------------------------|
| Understanding important tokens     | SHAP (or LIME for quick look)       |
| Justifying decisions with examples | Prototype (Text) / Grad-CAM (Image) |
| Fast, general-purpose insights     | LIME                                |
| In-depth audit or debugging        | SHAP                                |
| Stakeholder-friendly explanation   | Prototype / Grad-CAM                |

Combining multiple XAI methods provided the most complete understanding of model behavior. This approach helps ensure transparency and trustworthiness in real-world AI systems.

## Uses of LLM

- Used chatGPT4.0 while fine tuning the models for version control errors.
- Used for research in the area of XAI methods
- Used to study the dataset
- Used to Discover BERT-LORA while searching for a lighter model than BERT-UNCASED as we had a very small training Dataset and caused a lot of version problems for the transformer library that required us to fine tune BERT-UNCASED and use BERT-LORA instead.

## References

1. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). 'Why should I trust you?': Explaining the predictions of any classifier. KDD.
2. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. NeurIPS.
3. Kim, B., et al. (2014). Examples are not enough, learn to criticize! Criticism for interpretability.
4. Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). Deep Inside Convolutional Networks: Visualising Image Classification Models.
5. <https://www.kaggle.com/competitions/tweet-sentiment-extraction/data>
6. <https://github.com/acmi-lab/counterfactually-augmented-data/tree/master/sentiment>
7. [https://xaidataset.github.io/dataset/#face-glasses-recognition-\(glasses-xai\)](https://xaidataset.github.io/dataset/#face-glasses-recognition-(glasses-xai))

8. Used google colab GPU(T4) to fine tune the ML/DL model

# Team Contributions

---

Team Name: Encrypted\_Echoes

## **Dataset and Method Selection**

This was a collaborative effort contributed equally by all team members.

### **Dataset 1: Tweet Sentiment**

- Fine-tuning BERT: Manshul Budholiya (3851667)
- XAI Methods Explanation: Akshay Choudhary

### **Dataset 2: Movie Reviews (CAD - Sentiment Data)**

- Fine-tuning BERT with LoRA: Abdul Basit Baig (3851779)
- XAI Methods Explanation: Abdul Basit Baig (3851779)

### **Dataset 3: Glasses Detection (Image Dataset)**

- Fine-tuning ResNet18: Manshul Budholiya (3851667)
- XAI Methods Explanation: Manshul Budholiya (3851667)

## **Report and Presentation**

Prepared jointly and collaboratively by all team members.